

Assignment 2 : MicroServices

Benjamin Vandersmissen, Fabian Denoodt and José Oramas

DS 2024-2025

1 Introduction

This document provides an overview of the requirements and background information for the second practical assignment of the Distributed Systems course. The goal of this assignment is to familiarise yourself with the microservice design pattern, decompose a problem in microservices and implement microservices using Docker. You should solve the assignment **individually**.

2 Goal

The goal of this assignment is twofold:

- Firstly, you should decompose a scenario with different usecases in a microservice structure. This means designing different microservices, providing a description of the functionality that each microservice has, which databases it needs and the communications between microservices. You will then implement these with docker.
- Secondly, you will provide extensions to this project (related to the contents of this course) based on what you find interesting and would like to learn more about.

3 Base requirements

3.1 Microservices architecture

Given the following scenario, decompose in microservices in an intelligent way. This means grouping related functionality together in a single microservice. Pro-

vide an overview of the total microservice architecture in your report; this means which features each microservice implements, which databases a microservice has (if any) and which communication channels they have with the other microservices. In addition, you should provide your reasoning behind this decomposition.

Scenario: Suppose your best friend has a bright new idea for a video streaming service, that is guaranteed to generate billions of dollars. The goal is to fuse video streaming and social features into a single experience. Everything is prepared, even the name — NextFilm — and a logo. He only needs the following small set of features implemented.

1. A user can be created.
2. A user has the ability to add other users as friends.
3. A user has the ability to 'watch' a movie, putting it in its viewing history.
4. A user can rate a watched movie via a scoring system (based out of /10).
5. A user can view a friend's rated movies.
6. A user can 'agree' or 'disagree' with a rating by a friend.
7. A user can get recommendations based on top-rated movies.
8. A user can get recommendations based on movies seen by users friends, but not by the user.
9. A newsfeed exists in which the latest watched movies by the users friend are listed.
10. A full list of movies can retrieved.

Implement the microservice decomposition you made in python with Flask using Docker images (*except for requirement 10, which will be provided for you*). For each microservice you should ensure that the RESTful principles are followed, as well as provide a detailed overview of the endpoints and URI scheme in a manual, similar to Assignment 1. You should not have need of any external APIs.

To make things easier for you, you can use separate Docker containers for the databases. For example, if you decomposed the problem in two microservices *microA* and *microB*, you can implement two additional Docker containers *microA-db* and *microB-db*, but they should only contain databases associated with *microA*, *microB* respectively.

Internal (from *microA* to *microA-db*) communications *with a database* can be done using python libraries, while **external** (from *microA* to *microB*) communications between microservices need to be done via the REST protocol.

3.2 Python script

A non-GUI test script (`consume_script.py`) should consume your API and demonstrate that all functionalities listed in section 3.1 are implemented. A good testing script clearly indicates which functionality is currently being tested here, e.g. as follows:

```
# ** Functionality 1. Create a user **
print("this code tests this functionality ...")

# ** Functionality 2. ... **
...
```

4 Extension requirements

By only satisfying the base requirements, you can obtain a maximum score of 15/20. To obtain the remaining 5 points, you must propose and integrate meaningful extensions into your project. As a rule of thumb, the workload of these extensions should be approximately 25% of the base project. You are flexible in how you extend the project. Possible suggestions (for inspiration) include:

- Implementing advanced Docker features. These could include things like persistent storage, networks, ...
- Fault-resistant communication. Introducing techniques like health checks, or automatic restarts to handle cases where a microservice fails.
- Frontend integration. Develop a simple frontend using a modern library such as Svelte or React to interact with your API.
- Integrating database and API in a single (custom) docker image.

The main requirement for this part is that the extensions should contribute to your own learning. Simply implementing another basic functionality (e.g. extending the usecases of a specific microservice) adds little educational value and is therefore not considered a valid extension. The same applies to integrating technologies that have been covered in a previous course. You are now responsible for your own learning; enjoy it and make it worthwhile. You can always ask the teaching assistants during the lab sessions or via email for specific questions on valid extensions.

5 Deliverables

The following are the minimum requirements and deliverables for a passing grade:

- You should include a way for us to easily run your solution via the files `run_microservices.sh` and `consume_script.py`, which will automatically start your microservice architecture and run the Python test script, respectively. If necessary, you can use the `run_microservices.sh` file to populate the initial data required (such as movie names).
- Flask generates a local website containing the documentation of your API. However, you must also provide a document, `manual.pdf`, which contains a static copy of your documentation (e.g., obtained via "Print to PDF" in your browser). This is necessary in case your solution does not run on our devices, allowing us to still evaluate your work and assign meaningful scores.
- In addition, you should complete a report that discusses the following aspects:

Basic requirements

- Show the decomposition of the features in microservices, and argue why you have followed this decomposition.
- Discuss the connections of the different microservices. Do any microservices communicate with / depend on other microservices. What would happen if this communication is impeded?

Extension requirement (if applicable)

- Carefully discuss your extensions. Describe what you have added and why. If you implemented additional technologies or algorithms, explain what they do, and how they function. Note: your extensions will be primarily evaluated based on the report, so ensure that each extension is documented with sufficient depth.
- Ensure that your submitted solution is complete and contains no missing files. If you use additional libraries, provide us an easy way to install them (for Python, use a `requirements.txt` file).
- All solutions will be tested on the computers in the lab. These do not have `docker` installed, but rather `podman`. All commands of `docker` and `podman` are interchangeable, so make sure to only use `podman` commands.

6 Deadline and Submission

The deadline for this assignment is the **21st of May** at **23:59**. Late submissions will automatically have points deducted for tardiness.

Make sure that your submission has the following format and upload it via Blackboard. *DS-Assignment2-*Snumber-Lastname*.zip*.

Your submission should include at least the following items:

- run_microservices.sh
- consume_script.py
- report.md
- manual.pdf